

Web Application Security Part 2: XSS Analysis Lab

Penetration Testing Report

Name: solomon Mwaura

Date: April 15, 2026

Role: Junior Penetration Tester

Engagement Summary

The goal of this engagement was to evaluate the resilience of the HR platform's search and guestbook functions against **Reflected and Stored Cross-Site Scripting (XSS)** at **Medium** and **Impossible** security levels. I attempted to execute malicious JavaScript payloads to determine if sanitization or encoding measures were effective. While vulnerabilities were bypassable at lower levels, the "Impossible" level demonstrated robust defense-in-depth. This highlights that security is not a binary state but a progression of hardening techniques.

Testing Process Overview

I utilized a systematic methodology to verify security posture:

- **Modules:** Stored XSS (Guestbook) and Reflected XSS (Search).
 - **Level Management:** Security levels were toggled via the DVWA Security tab, followed by database resets to ensure a clean state.
 - **Tools:** **Burp Suite Community Edition** was used to intercept and modify HTTP POST/GET requests. **Firefox DevTools** and the **DVWA Source Viewer** were used to analyze DOM reflections and PHP source code logic.
 - **Confirmation:** Success was confirmed by the execution of a JavaScript `alert()`. Failure was confirmed by inspecting the page source to see if tags were stripped, modified, or safely encoded into HTML entities.
-

Findings Table

Module	Security Level	Payload Used	Result	Notes on Behavior / Output
Stored XSS	Low	<script>alert(1)</script>	Executed	Payload saved to DB and executes on every page load.
Stored XSS	Medium	<scr<script>ipt>alert(1)</script>	Executed	str_replace() removed the first tag; nested tag bypassed the filter.
Stored XSS	Impossible	<script>alert(1)</script>	Blocked	Payload stored as plain text; < converted to <.
Reflected XSS	Low	<script>alert(1)</script>	Executed	Input immediately reflected in the header tag.
Reflected XSS	Medium	<SCRIPT>alert(1)</SCRIPT>	Executed	str_replace only looked for lowercase; capitalized tags bypassed it.

Reflected XSS	Impossible	<code><script>alert(1)</script></code>	Blocked	<code>htmlspecialchars()</code> rendered the script harmlessly as text.
----------------------	-------------------	--	----------------	---

Mitigation Analysis

Target: Reflected XSS at **Impossible** Security Level.

- **What did the output look like?** The browser displayed the literal string `<script>alert(1)</script>` on the screen as text. No popup appeared, and the console showed no execution.
- **What code or filter appeared to block the script?** The PHP source code utilized the `htmlspecialchars()` function.
- **What security function(s) were used?** `htmlspecialchars($name)`. This function specifically converts special characters to HTML entities (e.g., `<` becomes `<`; and `>` becomes `>`).
- **Was the mitigation applied during input or output?** It was applied at the **output** stage, immediately before the variable was rendered in the HTML body.
- **Why was this approach effective?** It is effective because it breaks the "context" of the input. The browser no longer interprets the input as code (a tag) but as data (text). Even if an attacker bypasses input filters, proper output encoding ensures the payload remains inert in the user's browser.

Reflection

- **Behavioral Differences:** In the "Low" walkthrough, exploitation was trivial. At the "Medium" level, the application attempted "blacklisting" (stripping specific words). I found this to be a weak defense because it assumes the attacker will only use lowercase or simple tags. By using nested tags or capitalization, I could easily circumvent the blacklist.
- **Surprises:** I was surprised by how simple the "Impossible" fix is compared to the "Medium" fix. The Medium level used complex string replacement which failed, while the Impossible level used one standard PHP function (`htmlspecialchars`) that provided near-perfect protection. This reinforces the lesson that **Output Encoding** is far superior to **Input Filtering**.